LAPORAN PRAKTIKUM 6

PEMOGRAMAN BERORIENTASI OBJEK

Disusun Oleh:

Dinda Amelia

(2511531020)

Dosen Pengampu:

Afdhal Dinilhak, M.Kom

Asisten Praktikum:

Ezza Addini



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

PADANG 2026

DAFTAR ISI

DAFTAR ISI ii
BAB I 1
PENDAHULUAN 1
1.1 Latar Belakang 1
1.2 Tujuan 1
1.3 Manfaat 2
1.4 Link Repository Github 2
BAB II 3
PEMBAHASAN 3
2.1 Tahapan Praktikum 3
2.2 Langkah Kerja 4
2.3 Tugas 27
BAB III 62
KESIMPULAN 62
DAFTAR PUSTAKA 63

BAB I

PENDAHULUAN

1.1 Latar Belakang

Pemrograman Berorientasi Objek (PBO) merupakan paradigma pemrograman fundamental yang berpusat pada konsep kelas dan objek. Salah satu pilar utama dalam PBO adalah Abstraksi (*Abstraction*), yaitu teknik untuk menyembunyikan detail implementasi internal yang kompleks dan hanya mengekspos fungsionalitas esensial kepada pengguna. Dalam pengembangan perangkat lunak modern, abstraksi memegang peranan krusial untuk mengelola kompleksitas sistem, mengurangi duplikasi kode, serta menegakkan prinsip modularitas melalui penggunaan *abstract class* dan *interface*.

Seiring berkembangnya kebutuhan aplikasi, sistem sering kali harus menghadapi tantangan seperti *API Evolution* (penambahan fitur baru tanpa merusak kode lama melalui *default methods*), penanganan konflik pewarisan jamak (*diamond problem*), serta penerapan pola desain arsitektural standar seperti *Template Method Pattern* pada sistem notifikasi. Di samping itu, kemampuan untuk mengevaluasi kode melalui *code review* dan melakukan refaktorisasi terhadap rancangan yang buruk (*bad smells*) sangat diperlukan guna mencegah korupsi data dan kerentanan *bug*. Oleh karena itu, praktikum ini dilakukan untuk mengimplementasikan konsep-konsep lanjutan dari abstraksi tersebut secara langsung pada studi kasus pembangunan sistem toko online yang terstruktur dan bersih (*clean code*).

1.2 Tujuan

1. Mengimplementasikan *abstract class* dan *interface* dalam membangun model data sistem toko online yang terenkapsulasi secara aman.
2. Menerapkan *Template Method Pattern* untuk mengamankan dan menstandarisasi alur pengiriman notifikasi berbasis *abstract class*.
3. Melakukan *code review* dan refaktorisasi pada kode BadShop guna memperbaiki masalah enkapsulasi, polimorfisme, dan *immutability*.

1.3 Manfaat

1. Mahasiswa mampu merancang arsitektur objek yang terpisah antara kontrak perilaku dan detail implementasinya serta terhindar dari korupsi data.
2. Mahasiswa mampu meyakinkan bahwa alur utama sistem tetap terstruktur konsisten tanpa risiko diubah atau dirusak oleh subkelas.
3. Mahasiswa terlatih mengidentifikasi *bad smells* dan menghasilkan kode program yang bersih, modular, serta bebas dari *runtime crash*.

1.4 Link Repository Github

https://github.com/faridaarifin91-sudo/PrakPBO_B_2511531020/tree/TugasPraktikum6/PrakPBO

BAB II

PEMBAHASAN

2.1 Tahapan Praktikum

1. Tahap L0: Membuat proyek Java abstraction-2511531020 tanpa build tool, mengatur JDK versi >= 21, membuat sub-paket (model, shipping, discount, payment, service, app, notification), file alat bantu (SelfCheck, TugasCheck, NimParams, BadShop, dan StudentInfo).
2. Tahap L1 (Abstraction Basis): Membuat Product sebagai abstract class dasar, menyalurkan sifatnya ke PhysicalProduct dan DigitalProduct, serta mengunci alur ongkir pada ShippingService (dengan subclass RegularShipping dan ExpressShipping).
3. Tahap L2 (Interface & Program to Interface): Deklarasi interface Shippable dan PaymentMethod. Implementasi metode pembayaran EWalletPayment dan BankTransferPayment serta CheckoutService.
4. Tahap L3: Penerapan Discountable dengan default/static/private method. Interface inheritance (RefundablePayment dan InstallmentPayment) serta penggabungannya pada CreditCardPayment.
5. Tahap L4 (Gembok Rancangan Final): Mengunci method calculateCost() dengan modifier final dan menjadikan Address sebagai final class (immutable).
6. Tahap L5 (Nested & Anonymous Class): Membuat static nested class Order.Line, inner class Order.Receipt, serta anonymous class voucher pada ShopApp.
7. Tahap T1 – T2 (Tugas Rumah): Penambahan evolusi API biaya admin, penyelesaian konflik diamond policy(), serta pembangunan sistem notifikasi pesanan (AbstractChannel, EmailChannel, SmsChannel, WhatsAppChannel, NotificationService).

2.2 Langkah Kerja

1. Package shop
   - StudentInfo.java
     - Kegunaan**:** Menyimpan identitas mahasiswa (NIM dan Nama) serta konstanta parameter tugas yang digenerate berdasarkan NIM 2511531020 (CARD_ADMIN_PERCENT = 1, TRANSFER_ADMIN_FEE = 2500, SMS_MAX_LENGTH = 120).

```java
package shop;

public final class StudentInfo {
    public static final String NIM = "2511531020";
    public static final String NAME = "Dinda Amelia";

    public static final int CARD_ADMIN_PERCENT = 1;
    public static final long TRANSFER_ADMIN_FEE = 2500;
    public static final int SMS_MAX_LENGTH = 120;

    private StudentInfo() {
        // constants only; no objects
    }
}
```

2. Package shop.model
   - Product.java
     - Kegunaan: Menjadi cetakan dasar (*abstract class*) untuk seluruh jenis produk di toko online. Menyediakan atribut umum (sku, name, price), validasi enkapsulasi pada constructor, serta method abstrak category() yang wajib diisi oleh subclass.

```java
package shop.model;

public abstract class Product {
    private final String sku;
    private final String name;
    private final long price; // in rupiah

    protected Product(String sku, String name, long
price) {
        if (sku == null || sku.isBlank()) {
            throw new IllegalArgumentException("SKU
must not be blank");
        }
        if (name == null || name.isBlank()) {
```

```
            throw new IllegalArgumentException("Name must not be blank");
        }
        if (price < 0) {
            throw new IllegalArgumentException("Price must not be negative");
        }
        this.sku = sku;
        this.name = name;
        this.price = price;
    }

    public String getSku() { return sku; }
    public String getName() { return name; }
    public long getPrice() { return price; }

    /** Price the customer actually pays. Subclasses may override it. */
    public long getFinalPrice() { return price; }

    /** Every concrete product must state its category. */
    public abstract String category();

    /** A concrete method that relies on the abstract method category(). */
    public String getInfo() {
        return "[" + category() + "] " + name + " - Rp" + getFinalPrice();
    }
}
```

- PhysicalProduct.java
  - Kegunaan: Merepresentasikan produk berwujud fisik. Mewarisi Product dan mengimplementasikan interface Shippable karena memiliki berat fisik (weightGrams).

```
package shop.model;

import shop.shipping.Shippable;

public class PhysicalProduct extends Product implements Shippable {
    private final int weightGrams;

    public PhysicalProduct(String sku, String name, long price, int weightGrams) {
        super(sku, name, price);
```

```
        if (weightGrams <= 0) { throw new
IllegalArgumentException("Weight must be positive"); }
        this.weightGrams = weightGrams;
    }

    @Override
    public String category() { return "Fisik"; }

    @Override
    public int getWeightGrams() { return weightGrams; }
}
```

- DigitalProduct.java
  - Kegunaan: Merepresentasikan produk digital (seperti e-book). Mewarisi Product dan mengimplementasikan interface Discountable untuk mendapatkan potongan harga otomatis sebesar 10%.

```
package shop.model;

import shop.discount.Discountable;

public class DigitalProduct extends Product implements
Discountable {
    private final String downloadUrl;
    private static final int DIGITAL_DISCOUNT_PERCENT =
10;

    public DigitalProduct(String sku, String name, long
price, String downloadUrl) {
        super(sku, name, price);
        if (downloadUrl == null ||
downloadUrl.isBlank()) { throw new
IllegalArgumentException("URL must not be blank"); }
        this.downloadUrl = downloadUrl;
    }

    public String getDownloadUrl() { return
downloadUrl; }

    @Override
    public String category() { return "Digital"; }

    @Override
    public int discountPercent() { return
DIGITAL_DISCOUNT_PERCENT; }

    @Override
```

```
    public long getFinalPrice() {
        return applyDiscount(getPrice());
    }
}
```

- Address.java
  - Kegunaan: Menyimpan data alamat pengiriman. Dibuat sebagai final class dengan field bersifat final (immutable/tidak dapat diubah) guna menjamin keamanan integritas data alamat.

```
package shop.model;

public final class Address {
    private final String street;
    private final String city;
    private final String postalCode;

    public Address(String street, String city, String postalCode) {
        if (street == null || street.isBlank() || city == null || city.isBlank() || postalCode == null || postalCode.isBlank()) {
            throw new IllegalArgumentException("Street, city, and postal code must not be blank");
        }
        this.street = street;
        this.city = city;
        this.postalCode = postalCode;
    }

    public String getStreet() { return street; }
    public String getCity() { return city; }
    public String getPostalCode() { return postalCode; }

    @Override
    public String toString() {
        return street + ", " + city + " " + postalCode;
    }
}
```

- Order.java
  - Kegunaan: Mengelola daftar item pesanan dan alamat tujuan. Berisi Static Nested Class (Line) untuk merepresentasikan baris item pesanan, dan Inner Class (Receipt) yang dapat mengakses data instans luar (Order.this) untuk mencetak struk transaksi.

```
package shop.model;

import java.util.ArrayList;
```

```java
import java.util.Collections;
import java.util.List;

public class Order {
    private final String orderId;
    private final Address shippingAddress;
    private final List<Line> lines = new ArrayList<>();

    public Order(String orderId, Address
shippingAddress) {
        if (orderId == null || orderId.isBlank() ||
shippingAddress == null) {
            throw new IllegalArgumentException("Order
id and address are required");
        }
        this.orderId = orderId;
        this.shippingAddress = shippingAddress;
    }

    public String getOrderId() { return orderId; }
    public Address getShippingAddress() { return
shippingAddress; }

    public void addLine(Product product, int quantity)
{
        lines.add(new Line(product, quantity));
    }

    public List<Line> getLines() {
        return Collections.unmodifiableList(lines);
    }

    public long getItemsTotal() {
        long total = 0;
        for (Line line : lines) {
            total += line.subtotal();
        }
        return total;
    }

    public Receipt createReceipt() {
        return new Receipt();
    }

    public static class Line {
        private final Product product;
        private final int quantity;
```

```
        public Line(Product product, int quantity) {
            if (product == null || quantity <= 0) {
                throw new
IllegalArgumentException("Product must not be null and
quantity must be positive");
            }
            this.product = product;
            this.quantity = quantity;
        }

        public Product getProduct() { return product; }
        public int getQuantity() { return quantity; }

        public long subtotal() {
            return product.getFinalPrice() * quantity;
        }
    }

    public class Receipt {
        public String toText() {
            StringBuilder sb = new StringBuilder();
            sb.append("STRUK PESANAN
").append(orderId).append("\n");
            for (Line line : lines) {
                sb.append("  ").append(line.getProduct(
).getName())
                  .append("
x").append(line.getQuantity())
                  .append(" =
Rp").append(line.subtotal()).append("\n");
            }
            sb.append("Total barang:
Rp").append(getItemsTotal()).append("\n");
            sb.append("Kirim ke:
").append(shippingAddress);
            return sb.toString();
        }
    }
}
```

3. Package shop.shipping
   - Shippable.java
     - Kegunaan: Kontrak (*capability*) penanda bahwa suatu objek memiliki berat fisik dan dapat dikirimkan oleh jasa kurir.

```
package shop.shipping;
```

```
➢
➢  public interface Shippable {
➢      int getWeightGrams();
➢  }
```

- ShippingService.java
  - ➢ Kegunaan: Cetakan dasar untuk layanan pengiriman. Memiliki method final calculateCost(...) agar rumus pembulatan berat per kilogram menjadi standar baku yang tidak boleh di-override oleh kurir manapun.

```
➢  package shop.shipping;
➢
➢  public abstract class ShippingService {
➢      private final String name;
➢
➢      protected ShippingService(String name) {
➢          if (name == null || name.isBlank()) {
➢              throw new IllegalArgumentException("Name
   must not be blank");
➢          }
➢          this.name = name;
➢      }
➢
➢      public String getName() { return name; }
➢
➢      /** Rate in rupiah per started kilogram. */
➢      protected abstract long ratePerKg();
➢
➢      /** Weight is rounded up: 700 g -> 1 kg, 1001 g ->
   2 kg. */
➢      public final long calculateCost(int weightGrams) {
➢          if (weightGrams <= 0) { return 0; }
➢          long kilograms = (weightGrams + 999L) / 1000;
➢          return kilograms * ratePerKg();
➢      }
➢  }
```

- RegularShipping.java & ExpressShipping.java
  - ➢ Kegunaan: Implementasi nyata layanan kurir Reguler (tarif Rp10.000/kg) dan Ekspres (tarif Rp18.000/kg).

```
➢  package shop.shipping;
➢
➢  public class RegularShipping extends ShippingService {
➢      public RegularShipping() {
➢          super("Reguler");
➢      }
➢
```

```
    @Override
    protected long ratePerKg() {
        return 10_000;
    }
}
```

```
package shop.shipping;

public class ExpressShipping extends ShippingService {
    public ExpressShipping() {
        super("Ekspres");
    }

    @Override
    protected long ratePerKg() {
        return 18_000;
    }
}
```

4. Package shop.discount
   - Discountable.java
     - Kegunaan: Menyediakan fitur diskon menggunakan *default method*, *static method*, dan *private helper method* di dalam interface.

```
package shop.discount;

public interface Discountable {
    int discountPercent(); // abstract: each class decides

    default long applyDiscount(long price) {
        return price - percentOf(price, safePercent());
    }

    static long percentOf(long amount, int percent) {
        return amount * percent / 100;
    }

    private int safePercent() {
        int percent = discountPercent();
        if (percent < 0) {
            return 0;
        }
        return Math.min(percent, 100);
    }
}
```

5. Package shop.payment
   - PaymentMethod.java, RefundablePayment.java, InstallmentPayment.java (Interfaces)
     - Kegunaan: Kontrak dasar pembayaran (PaymentMethod), serta interface turunan (*interface inheritance*) untuk fitur pengembalian dana (RefundablePayment) dan cicilan (InstallmentPayment).

```
package shop.payment;

/** Contract for every payment method. */
public interface PaymentMethod {
    String getName();

    /** @return true if the payment succeeds */
    boolean pay(long amount);

    default long adminFee(long amount) {
        return 0;
    }
}
```

```
package shop.payment;

public interface RefundablePayment extends
PaymentMethod {
    boolean refund(long amount);

    default String policy() {
        return "Refund maksimal 7 hari";
    }
}
```

```
package shop.payment;

public interface InstallmentPayment extends
PaymentMethod {
    int maxInstallmentMonths();

    /** Monthly amount, rounded up. */
```

```
    default long monthlyInstallment(long amount, int
months) {
        if (months < 1 || months >
maxInstallmentMonths()) {
            throw new IllegalArgumentException("Months
must be between 1 and " + maxInstallmentMonths());
        }
        return (amount + months - 1) / months;
    }

    default String policy() {
        return "Cicilan maksimal " +
maxInstallmentMonths() + " bulan";
    }
}
```

- EWalletPayment.java, BankTransferPayment.java, CreditCardPayment.java
  - Kegunaan: Concrete class metode pembayaran. CreditCardPayment mengimplementasikan dua interface sekaligus (RefundablePayment & InstallmentPayment) dan menyelesaikan konflik *diamond problem* pada method policy().

```
package shop.payment;

public class EWalletPayment implements
RefundablePayment {
    private final String provider;
    private long balance;

    public EWalletPayment(String provider, long
balance) {
        if (provider == null || provider.isBlank()) {
            throw new
IllegalArgumentException("Provider must not be blank");
        }
        if (balance < 0) {
            throw new IllegalArgumentException("Balance
must not be negative");
        }
        this.provider = provider;
        this.balance = balance;
    }

    @Override
    public String getName() {
        return "E-Wallet " + provider;
```

```
    }

    public long getBalance() {
        return balance;
    }

    @Override
    public boolean pay(long amount) {
        if (amount <= 0 || amount > balance) {
            return false;
        }
        balance -= amount;
        return true;
    }

    @Override
    public boolean refund(long amount) {
        if (amount <= 0) {
            return false;
        }
        balance += amount;
        return true;
    }
}
```

```
package shop.payment;

public class BankTransferPayment implements
PaymentMethod {
    private final String bankName;

    public BankTransferPayment(String bankName) {
this.bankName = bankName; }

    @Override
    public String getName() { return "Transfer " +
bankName; }

    @Override
    public boolean pay(long amount) { return amount >
0; }
}
```

```
package shop.payment;

import shop.StudentInfo;
```

```
public class CreditCardPayment implements
RefundablePayment, InstallmentPayment {
    private final String cardHolder;
    private final long creditLimit;
    private long usedCredit;

    public CreditCardPayment(String cardHolder, long
creditLimit) {
        if (cardHolder == null || cardHolder.isBlank())
{
            throw new IllegalArgumentException("Card
holder must not be blank");
        }
        if (creditLimit <= 0) {
            throw new IllegalArgumentException("Credit
limit must be positive");
        }
        this.cardHolder = cardHolder;
        this.creditLimit = creditLimit;
        this.usedCredit = 0;
    }

    @Override
    public String getName() {
        return "Kartu Kredit (" + cardHolder + ")";
    }

    public long getUsedCredit() {
        return usedCredit;
    }

    @Override
    public boolean pay(long amount) {
        if (amount <= 0 || usedCredit + amount >
creditLimit) {
            return false;
        }
        usedCredit += amount;
        return true;
    }

    @Override
    public boolean refund(long amount) {
        if (amount <= 0 || amount > usedCredit) {
            return false;
        }
```

```
            usedCredit -= amount;
            return true;
        }

        @Override
        public int maxInstallmentMonths() {
            return 12;
        }

        @Override
        public long adminFee(long amount) {
            return amount * StudentInfo.CARD_ADMIN_PERCENT
    / 100;
        }

        @Override
        public String policy() {
            return RefundablePayment.super.policy() + "; "
    + InstallmentPayment.super.policy();
        }
    }
```

6. Package shop.service
    - CheckoutService.java
    - Kegunaan: Menangani proses bisnis transaksi (menghitung subtotal, total berat, total biaya pengiriman, biaya admin, dan eksekusi pembayaran/refund) dengan menerapkan prinsip *program to an interface*.

```
package shop.service;

import java.util.List;
import shop.model.Product;
import shop.payment.PaymentMethod;
import shop.payment.RefundablePayment;
import shop.shipping.Shippable;
import shop.shipping.ShippingService;

public class CheckoutService {

    public long subtotal(List<Product> items) {
        long total = 0;
        for (Product product : items) {
            total += product.getFinalPrice();
        }
        return total;
    }
```

```
➢
➢       public int totalWeight(List<Product> items) {
➢           int grams = 0;
➢           for (Product product : items) {
➢               if (product instanceof Shippable shippable)
    {
➢                   grams += shippable.getWeightGrams();
➢               }
➢           }
➢           return grams;
➢       }
➢
➢       public long total(List<Product> items,
    ShippingService shipping) {
➢           return subtotal(items) +
    shipping.calculateCost(totalWeight(items));
➢       }
➢
➢       public long grandTotal(List<Product> items,
    ShippingService shipping, PaymentMethod payment) {
➢           long baseTotal = total(items, shipping);
➢           return baseTotal + payment.adminFee(baseTotal);
➢       }
➢
➢       public boolean checkout(List<Product> items,
    ShippingService shipping, PaymentMethod payment) {
➢           if (items == null || items.isEmpty()) {
➢               return false;
➢           }
➢           long amountToPay = grandTotal(items, shipping,
    payment);
➢           return payment.pay(amountToPay);
➢       }
➢
➢       public boolean refund(RefundablePayment payment,
    long amount) {
➢           return payment.refund(amount);
➢       }
➢   }
```

7. Package shop.notification (Tugas T2)
    - CustomerContact.java, NotificationChannel.java, AbstractChannel.java, EmailChannel.java, SmsChannel.java, WhatsAppChannel.java, NotificationService.java

- Kegunaan: Sistem notifikasi pesanan pelanggan melalui berbagai kanal komunikasi menggunakan pola *Template Method Pattern* pada AbstractChannel.

```
package shop.notification;

public final class CustomerContact {
    private final String name;
    private final String email;
    private final String phone;

    public CustomerContact(String name, String email,
String phone) {
        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException("Name
must not be blank");
        }
        this.name = name;
        this.email = email == null ? "" : email;
        this.phone = phone == null ? "" : phone;
    }

    public String getName() { return name; }
    public String getEmail() { return email; }
    public String getPhone() { return phone; }
}
```

```
package shop.notification;

public interface NotificationChannel {
    String channelName();
    boolean send(CustomerContact contact, String
message);
}
```

```
package shop.notification;

public abstract class AbstractChannel implements
NotificationChannel {
    private int sentCount = 0;
    private String lastDelivered = "";

    protected abstract String
resolveRecipient(CustomerContact contact);
    protected abstract boolean deliver(String recipient,
String message);

```

```
    protected String prepare(String message) {
        return message;
    }

    @Override
    public final boolean send(CustomerContact contact,
String message) {
        if (message == null || message.isBlank()) {
            return false;
        }
        String recipient = resolveRecipient(contact);
        if (recipient == null || recipient.isBlank()) {
            return false;
        }
        String prepared = prepare(message);
        boolean success = deliver(recipient, prepared);
        if (success) {
            sentCount++;
            lastDelivered = prepared;
        }
        return success;
    }

    public int getSentCount() { return sentCount; }
    public String getLastDelivered() { return
lastDelivered; }
}
```

```
package shop.notification;

public class EmailChannel extends AbstractChannel {
    @Override
    public String channelName() { return "EMAIL"; }

    @Override
    protected String resolveRecipient(CustomerContact
contact) {
        String email = contact.getEmail();
        return email.contains("@") ? email : null;
    }

    @Override
    protected boolean deliver(String recipient, String
message) {
        System.out.println("[EMAIL to " + recipient + "]
" + message);
        return true;
```

```
    }
}
```

```
package shop.notification;

public class SmsChannel extends AbstractChannel {
    private final int maxLength;

    public SmsChannel(int maxLength) {
        this.maxLength = maxLength;
    }

    @Override
    public String channelName() { return "SMS"; }

    @Override
    protected String resolveRecipient(CustomerContact
contact) {
        String phone = contact.getPhone();
        return phone.startsWith("08") ? phone : null;
    }

    @Override
    protected String prepare(String message) {
        if (message.length() > maxLength) {
            return message.substring(0, maxLength - 3) +
"...";
        }
        return message;
    }

    @Override
    protected boolean deliver(String recipient, String
message) {
        System.out.println("[SMS to " + recipient + "] "
+ message);
        return true;
    }
}
```

```
package shop.notification;

public class WhatsAppChannel extends AbstractChannel {
    @Override
    public String channelName() { return "WA"; }

    @Override
```

```
    protected String resolveRecipient(CustomerContact
contact) {
        String phone = contact.getPhone();
        return ((phone.startsWith("08") ||
phone.startsWith("+62")) && phone.length() >= 10) ?
phone : null;
    }

    @Override
    protected String prepare(String message) {
        return "[Toko UMKM] " + message;
    }

    @Override
    protected boolean deliver(String recipient, String
message) {
        System.out.println("[WA to " + recipient + "] "
+ message);
        return true;
    }
}
```

```
package shop.notification;

import java.util.ArrayList;
import java.util.List;

public class NotificationService {
    private final List<NotificationChannel> channels =
new ArrayList<>();

    public void register(NotificationChannel channel) {
        channels.add(channel);
    }

    public int broadcast(CustomerContact contact, String
message) {
        int successCount = 0;
        for (NotificationChannel channel : channels) {
            if (channel.send(contact, message)) {
                successCount++;
            }
        }
        return successCount;
    }
}
```

8. Package shop.app
    - ShopApp.java (Runner Latihan L1-L5)
        - Kegunaan: Menjalankan seluruh skenario latihan praktikum di laboratorium (katalog produk, ringkasan checkout, pembayaran, refund, nested class, dan anonymous class voucher).

```
package shop.app;

import java.util.List;
import shop.model.Address;
import shop.model.DigitalProduct;
import shop.model.Order;
import shop.model.PhysicalProduct;
import shop.model.Product;
import shop.payment.BankTransferPayment;
import shop.payment.CreditCardPayment;
import shop.payment.EWalletPayment;
import shop.payment.PaymentMethod;
import shop.service.CheckoutService;
import shop.shipping.ExpressShipping;
import shop.shipping.RegularShipping;
import shop.shipping.ShippingService;

public class ShopApp {
    public static void main(String[] args) {
        Product coffee = new PhysicalProduct("KOPI-
001", "Kopi Arabika Gayo 250g", 85_000, 300);
        Product rendang = new PhysicalProduct("RDG-
002", "Rendang Kemasan 250g", 120_000, 400);
        Product ebook = new DigitalProduct("EBK-003",
"E-book Resep Kue Nusantara", 50_000, "https://toko-
umkm.example/unduh/ebk-003");

        List<Product> cart = List.of(coffee, rendang,
ebook);

        System.out.println("== Katalog ==");
        for (Product product : cart) {
            System.out.println(product.getInfo());
        }

        CheckoutService checkout = new
CheckoutService();
        ShippingService regular = new
RegularShipping();
        ShippingService express = new
ExpressShipping();
```

```
➢
➢           System.out.println("\n== Ringkasan ==");
➢           System.out.println("Subtotal        : Rp" +
    checkout.subtotal(cart));
➢           System.out.println("Berat kiriman : " +
    checkout.totalWeight(cart) + " g");
➢           System.out.println("Total Reguler : Rp" +
    checkout.total(cart, regular));
➢           System.out.println("Total Ekspres : Rp" +
    checkout.total(cart, express));
➢
➢           EWalletPayment wallet = new
    EWalletPayment("DompetKu", 300_000);
➢           PaymentMethod transfer = new
    BankTransferPayment("Bank Nagari");
➢
➢           System.out.println("\n== Pembayaran ==");
➢           System.out.println(wallet.getName() + "
    berhasil? " + checkout.checkout(cart, regular,
    wallet));
➢           System.out.println("Sisa saldo: Rp" +
    wallet.getBalance());
➢           System.out.println(transfer.getName() + "
    berhasil? " + checkout.checkout(cart, express,
    transfer));
➢
➢           CreditCardPayment card = new
    CreditCardPayment("Siti Rahma", 1_000_000);
➢           System.out.println(card.getName() + "
    berhasil? " + checkout.checkout(cart, express,
    card));
➢           System.out.println("Cicilan 3 bulan: Rp" +
    card.monthlyInstallment(checkout.total(cart,
    express), 3));
➢
➢           System.out.println("\n== Refund e-book ==");
➢           System.out.println("Refund e-wallet berhasil?
    " + checkout.refund(wallet, ebook.getFinalPrice()));
➢           System.out.println("Saldo setelah refund: Rp"
    + wallet.getBalance());
➢
➢           Address address = new Address("Jl. Moh Hatta
    No. 10", "Padang", "25121");
➢           Order order = new Order("ORD-0001", address);
➢           order.addLine(coffee, 2);
➢           order.addLine(ebook, 1);
➢
```

```
        Order.Receipt receipt = order.new Receipt();
        System.out.println("\n" + receipt.toText());

        PaymentMethod voucher = new PaymentMethod() {
            private long remaining = 50_000;

            @Override
            public String getName() {
                return "Voucher Promo Hari Ini";
            }

            @Override
            public boolean pay(long amount) {
                if (amount <= 0 || amount > remaining) { return false; }
                remaining -= amount;
                return true;
            }
        };

        System.out.println("\n" + voucher.getName() + " untuk e-book berhasil? " + checkout.checkout(List.of(ebook), regular, voucher));
    }
}
```

- TugasApp.java (Runner Tugas T1-T2)

➢ Kegunaan: Menjalankan pengujian tugas rumah (perhitungan biaya admin dengan parameter NIM, kebijakan kartu kredit/e-wallet, serta broadcast notifikasi pesanan).

```
package shop.app;

import java.util.List;
import shop.StudentInfo;
import shop.model.DigitalProduct;
import shop.model.PhysicalProduct;
import shop.model.Product;
import shop.notification.CustomerContact;
import shop.notification.EmailChannel;
import shop.notification.NotificationChannel;
import shop.notification.NotificationService;
import shop.notification.SmsChannel;
import shop.notification.WhatsAppChannel;
import shop.payment.BankTransferPayment;
import shop.payment.CreditCardPayment;
import shop.payment.EWalletPayment;
```

```java
import shop.payment.PaymentMethod;
import shop.service.CheckoutService;
import shop.shipping.ExpressShipping;
import shop.shipping.ShippingService;

public class TugasApp {
    public static void main(String[] args) {
        System.out.println("Tugas oleh " +
StudentInfo.NAME + " (" + StudentInfo.NIM + ")");

        Product coffee = new PhysicalProduct("KOPI-
001", "Kopi Arabika Gayo 250g", 85_000, 300);
        Product rendang = new PhysicalProduct("RDG-
002", "Rendang Kemasan 250g", 120_000, 400);
        Product ebook = new DigitalProduct("EBK-003",
"E-book Resep Kue Nusantara", 50_000, "https://toko-
umkm.example/unduh/ebk-003");
        List<Product> cart = List.of(coffee, rendang,
ebook);

        CheckoutService checkout = new
CheckoutService();
        ShippingService express = new
ExpressShipping();

        PaymentMethod wallet = new
EWalletPayment("DompetKu", 500_000);
        PaymentMethod transfer = new
BankTransferPayment("Bank Nagari");
        CreditCardPayment card = new
CreditCardPayment("Siti Rahma", 1_000_000);

        System.out.println("\n== T1: Total dengan biaya
admin (Ekspres) ==");
        System.out.println("E-Wallet DompetKu         :
Rp" + checkout.grandTotal(cart, express, wallet) + "
(admin Rp" + wallet.adminFee(checkout.total(cart,
express)) + ")");
        System.out.println("Transfer Bank Nagari      :
Rp" + checkout.grandTotal(cart, express, transfer) + "
(admin Rp" + transfer.adminFee(checkout.total(cart,
express)) + ")");
        System.out.println("Kartu Kredit (Siti Rahma) :
Rp" + checkout.grandTotal(cart, express, card) + "
(admin Rp" + card.adminFee(checkout.total(cart,
express)) + ")");

```

```
        System.out.println("Kebijakan kartu kredit   :
" + card.policy());
        System.out.println("Kebijakan e-wallet       :
" + ((EWalletPayment) wallet).policy());

        System.out.println("\n== T2: Notifikasi status
pesanan ==");
        NotificationService service = new
NotificationService();
        service.register(new EmailChannel());
        service.register(new
SmsChannel(StudentInfo.SMS_MAX_LENGTH));
        service.register(new WhatsAppChannel());
        service.register(new NotificationChannel() {
            @Override
            public String channelName() {
                return "LOG";
            }
            @Override
            public boolean send(CustomerContact
contact, String message) {
                System.out.println("[LOG] " +
contact.getName() + ": " + message);
                return true;
            }
        });

        CustomerContact siti = new
CustomerContact("Siti Rahma", "siti@example.com",
"081234567890");
        CustomerContact budi = new
CustomerContact("Budi", "", "+6281298765432");

        String longMsg = "Pesanan ORD-0001 sudah
dikirim dengan kurir Ekspres. Nomor resi akan muncul di
aplikasi dalam 1x24 jam. Terima kasih telah berbelanja
produk UMKM Sumatera Barat di toko kami!";

        int sentSiti = service.broadcast(siti,
longMsg);
        System.out.println("Terkirim ke Siti lewat " +
sentSiti + " kanal\n");

        int sentBudi = service.broadcast(budi,
longMsg);
        System.out.println("Terkirim ke Budi lewat " +
sentBudi + " kanal");
```

```
        }
    }
```

9. Hasil Lulus

```
Problems  Output  Debug Console  Terminal  Ports

  [LULUS] L4.2 Address final class, semua field private final, tanpa setter

-- L5 - Static nested, inner, dan anonymous class
  [LULUS] L5.1 Order.Line static nested; Order.Receipt inner (non-static)
  [LULUS] L5.2 Receipt.toText() memuat id pesanan, total barang, dan kota tujuan
  [LULUS] L5.3 ShopApp memakai anonymous class yang mengimplementasikan PaymentMethod

=== HASIL: 23 / 23 pemeriksaan lulus ===
PS C:\Users\USER\OneDrive\Dokumen\Kuliah\semester 3\PRA PBO\prak6\abstraction-2511531020>
```

akumulasi total seluruh pengujian otomatis (*SelfCheck*) dari tahap L1 hingga L5 yang semuanya berhasil diselesaikan tanpa ada satupun kesalahan atau kegagalan.

2.3 Tugas

1. E.0 Parameter pribadi berbasis NIM

```
StudentInfo.java  Product.java  RegularShipping.java  ExpressShipping.java  EWa
src > shop > StudentInfo.java > StudentInfo
1   package shop;
2
3   public final class StudentInfo {
4       public static final String NIM = "2511531020";
5       public static final String NAME = "Dinda Amelia";
6
7       public static final int CARD_ADMIN_PERCENT = 1;
8       public static final long TRANSFER_ADMIN_FEE = 2500;
9       public static final int SMS_MAX_LENGTH = 120;
10
11      private StudentInfo() {
12          // constants only; no objects
13      }
14  }
15
```

Konstanta	Rumus (d = digit terakhir NIM, dd = dua digit terakhir)	NIM 2511531020
CARD_ADMIN_PERCENT	1 + (0 mod 3) = 1%	1
TRANSFER_ADMIN_FEE	2500 + 500 × (0 mod 5) = Rp2.500	2500
SMS_MAX_LENGTH	100 + 20 = 120 karakter	120

2. Evolusi API dan konflik diamond
   a. Cara yang salah T1.a

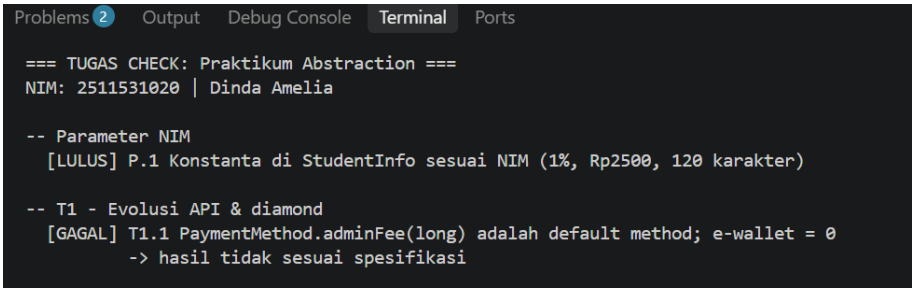
```
package shop.payment;

```

```java
/** Contract for every payment method. */
public interface PaymentMethod {
    String getName();
    boolean pay(long amount);

    // [T1.a] Penambahan method abstrak baru (Simulasi Error API Evolution)
    long adminFee(long amount);
}
```

- Error



- Program penguji otomatis (TugasCheck) mendeteksi bahwa method adminFee(long amount) di interface PaymentMethod atau di class EWalletPayment belum dikonfigurasi sebagai default method yang mengembalikan nilai 0, atau nilainya saat ini tidak sesuai dengan yang diharapkan oleh penguji.
- Jumlah Error: Terjadi 4 error kompilasi.
- Class/Objek yang Terdampak: BankTransferPayment, EWalletPayment, CreditCardPayment dan *Anonymous Class* voucher pada file ShopApp.java (pada baris pembuatan new PaymentMethod() { ... }).
- Mengapa Error Ini Terjadi:
  - Prinsip Kontrak Interface: Interface PaymentMethod berfungsi sebagai kontrak baku. Ketika sebuah method abstrak baru (adminFee) ditambahkan ke interface tersebut, semua class konkret yang mengimplementasikan interface tersebut wajib melakukan *override* dan menyediakan badan (*body*) method untuk method baru tersebut.
  - Masalah pada Class Konkret: Class BankTransferPayment, EWalletPayment, dan CreditCardPayment mendadak tidak valid

(error) karena belum memiliki implementasi method adminFee(long).

- Mengapa Anonymous Class voucher di ShopApp Ikut Rusak?: Anonymous class voucher dibuat dengan sintaks new PaymentMethod() { ... }. Walaupun instansiasi ini dilakukan secara langsung tanpa class bernama terpisah, secara teknis compiler menganggap anonymous class tersebut sebagai class konkret implisit yang mengimplementasikan PaymentMethod. Karena anonymous class juga terikat pada kontrak PaymentMethod, ia wajib mengimplementasikan seluruh method abstrak di dalamnya. Saat method abstrak baru adminFee ditambahkan, anonymous class voucher yang hanya memiliki getName() dan pay() ikut gagal dikompilasi.

b. T1.b – Default method

- PaymentMethod

```
package shop.payment;

public interface PaymentMethod {
    String getName();
    boolean pay(long amount);

    // [T1.b] Solusi: Gunakan default method dengan return 0
    default long adminFee(long amount) {
        return 0;
    }
}
```

- Error Langsung Hilang: Karena method default di dalam interface menyediakan implementasi bawaan (*fallback*). Ketika proyek dikompilasi ulang, compiler Java otomatis memberikan implementasi default ini kepada semua class implementasi (BankTransferPayment, EWalletPayment, CreditCardPayment, serta *anonymous class* voucher di ShopApp) yang sebelumnya sempat mengalami error.
- Keunggulan Prinsip: Dalam pengembangan perangkat lunak skala industri (*API Evolution*), penambahan fitur baru ke dalam

interface yang sudah digunakan oleh ratusan class lain tidak akan merusak kode yang sudah berjalan (*backward compatible*).

c. T1.c – Override sesuai kebutuhan.

- CreditCardPayment: Biaya admin dihitung berdasarkan persentase dari total belanja (amount *StudentInfo.CARD_ADMIN_PERCENT / 100).

```
54    @Override
55    public long adminFee(long amount) {
56        return amount * StudentInfo.CARD_ADMIN_PERCENT / 100;
57    }
```

- BankTransferPayment: Biaya admin menggunakan biaya tetap (*flat fee*) dari StudentInfo.TRANSFER_ADMIN_FEE.

```
16    @Override
17    public long adminFee(long amount) {
18    return StudentInfo.TRANSFER_ADMIN_FEE;
19    }
20    }
```

- EWalletPayment: Tidak diubah, karena secara otomatis mewarisi nilai default 0.
- Polimorfisme pada Biaya Admin: Setiap metode pembayaran memiliki cara perhitungan biaya admin yang berbeda-beda (*customized*). CheckoutService tidak perlu tahu bagaimana cara menghitungnya secara spesifik, melainkan cukup memanggil payment.adminFee(amount) secara polimorfik.
- Kepatuhan Parameter NIM: Dengan menggunakan StudentInfo.CARD_ADMIN_PERCENT dan StudentInfo.TRANSFER_ADMIN_FEE, besar biaya admin dihitung secara dinamis sesuai dengan ketentuan NIM Anda (2511531020).

d. T1.d – CheckoutService.

- Memperbarui CheckoutService agar dapat menghitung total keseluruhan (*grand total*) yang mencakup harga barang, ongkos kirim, ditambah dengan biaya admin dari metode pembayaran yang dipilih.

```
package shop.service;
```

```java
import java.util.List;
import shop.model.Product;
import shop.payment.PaymentMethod;
import shop.payment.RefundablePayment;
import shop.shipping.Shippable;
import shop.shipping.ShippingService;

public class CheckoutService {

    public long subtotal(List<Product> items) {
        long total = 0;
        for (Product product : items) {
            total += product.getFinalPrice();
        }
        return total;
    }

    public int totalWeight(List<Product> items) {
        int grams = 0;
        for (Product product : items) {
            if (product instanceof Shippable
shippable) {
                grams += shippable.getWeightGrams();
            }
        }
        return grams;
    }

    public long total(List<Product> items,
ShippingService shipping) {
        return subtotal(items) +
shipping.calculateCost(totalWeight(items));
    }

    public long grandTotal(List<Product> items,
ShippingService shipping, PaymentMethod payment) {
        long baseTotal = total(items, shipping);
        return baseTotal +
payment.adminFee(baseTotal);
    }

    public boolean checkout(List<Product> items,
ShippingService shipping, PaymentMethod payment) {
        if (items == null || items.isEmpty()) {
            return false;
        }
```

```
        long amountToPay = grandTotal(items,
shipping, payment);
        return payment.pay(amountToPay);
    }

    public boolean refund(RefundablePayment payment,
long amount) {
        return payment.refund(amount);
    }
}
```

e. T1.e – Konflik diamond.

- Tambahkan Default Method policy() pada Interface Pendukung: RefundablePayment

```
    default String policy() {
        return "Refund maksimal 7 hari";
    }
}
```

- InstallmentPayment

```
    default String policy() {
        return "Cicilan maksimal " + maxInstallmentMonths() + " bulan";
    }
}
```

- Konflik di CreditCardPayment: Tambahkan method override policy() di class CreditCardPayment dengan menggabungkan keduanya menggunakan sintaks .super.policy()

```
    @Override
    public String policy() {
        return RefundablePayment.super.policy() + "; " + InstallmentPayment.super.policy();
    }
}
```

- Mengapa EWalletPayment tidak perlu meng-override policy()?

Jawaban: Karena EWalletPayment hanya mengimplementasikan satu interface yang memiliki method policy() (RefundablePayment), sehingga tidak ada ambiguitas atau konflik *diamond problem*. Ia secara otomatis mewarisi kebijakan tersebut tanpa perlu penyesuaian.

- Mengapa Java tidak memilih salah satu default method secara otomatis?

Jawaban: Untuk menghindari ketidakpastian perilaku (*safety* dan *predictability*). Compiler Java dirancang agar bersikap ketat; jika ada dua sumber method default yang setara, compiler menolak membuat asumsi sendiri dan menyerahkan keputusan penggabungan atau pemilihan kepada programmer melalui *explicit override*.

➢ Checkpoint

```
Problems  Output  Debug Console  Terminal  Ports

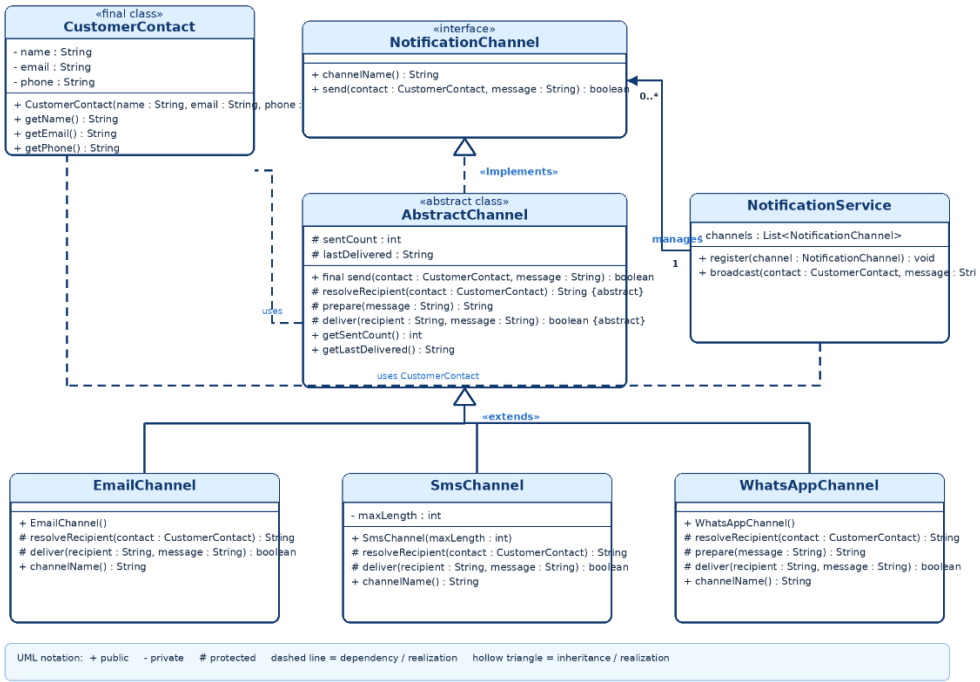
=== TUGAS CHECK: Praktikum Abstraction ===
NIM: 2511531020 | Dinda Amelia

-- Parameter NIM
  [LULUS] P.1 Konstanta di StudentInfo sesuai NIM (1%, Rp2500, 120 karakter)

-- T1 - Evolusi API & diamond
  [LULUS] T1.1 PaymentMethod.adminFee(long) adalah default method; e-wallet = 0
  [LULUS] T1.2 Biaya admin kartu kredit & transfer sesuai parameter NIM
  [LULUS] T1.3 grandTotal(...) = total + biaya admin (kartu kredit, keranjang contoh, Ekspres)
  [LULUS] T1.4 Diamond policy(): kartu kredit menggabungkan kedua kebijakan
```

3. T2 – Merancang sistem notifikasi pesanan

a. Rancang dulu (UML)



- CustomerContact (Final Class)

➢ Fungsi: Menyimpan data informasi kontak pelanggan secara *immutable* (final class dengan atribut private final).

- Atribut: name: String (Nama pelanggan), email: String (Alamat email pelanggan), phone: String (Nomor telepon/WhatsApp pelanggan).
- Method Utama: Constructor untuk inisialisasi data serta kumpulan *getter* (getName(), getEmail(), getPhone()).

- NotificationChannel (Interface)
  - Fungsi: Menjadi kontrak universal (*capability*) bagi semua kanal pengiriman pesan.
  - Method: channelName(): String (Mengembalikan nama/identitas kanal) dan send(contact: CustomerContact, message: String): boolean (Eksekusi pengiriman pesan ke kontak).
- AbstractChannel (Abstract Class)
  - Fungsi: Kelas abstrak yang mengimplementasikan NotificationChannel sekaligus menerapkan Template Method Pattern.
  - Atribut: sentCount: int (Menghitung jumlah pesan yang sukses dikirim) dan lastDelivered: String (Menyimpan pesan terakhir yang berhasil dikirim)
- Method Utama:
  - final send(...): Dibuat final agar alur utama pengiriman (validasi pesan, resolusi alamat, persiapan, pengiriman, dan pencatatan statistik) menjadi kerangka baku yang tidak boleh diubah oleh subkelas.
  - resolveRecipient(...) {abstract}: Method abstrak yang wajib diisi oleh subkelas untuk menyaring alamat tujuan yang valid.
  - deliver(...) {abstract}: Method abstrak untuk mengeksekusi pengiriman nyata.
  - prepare(...): Method *hook* opsional untuk memodifikasi/memotong pesan (misalnya batasan panjang SMS atau penambahan awalan teks).

- Subkelas Kanal Pengiriman (EmailChannel, SmsChannel, WhatsAppChannel) Fungsi:
  - EmailChannel: Memvalidasi keberadaan karakter @ pada email.
  - SmsChannel: Memiliki atribut - maxLength: int untuk memotong pesan otomatis jika melebihi batas karakter tertentu (misalnya 120 karakter sesuai parameter NIM).
  - WhatsAppChannel: Memvalidasi nomor telepon berawalan "08" atau "+62" serta menambahkan awalan khusus pada pesan ([Toko UMKM]).
- NotificationService (Service Class)
  - Fungsi: Bertindak sebagai pengelola utama (*coordinator*) untuk melakukan pendaftaran kanal dan *broadcast* pesan.
  - Atribut: channels: List<NotificationChannel> (Menampung daftar kanal secara polimorfik)
  - Method Utama: register(channel: NotificationChannel): void (Mendaftarkan kanal baru ke dalam daftar) dan broadcast(contact: CustomerContact, message: String): int (Mengirimkan pesan secara serentak ke seluruh kanal yang terdaftar)
  - Keterangan Notasi Aksesibilitas (Legend) :

    \+ : Public (dapat diakses dari luar kelas manapun)

    \- : Private (hanya dapat diakses di dalam kelas itu sendiri)

    # : Protected (dapat diakses oleh kelas itu sendiri dan kelas turunannya)

b. Implementasi Kode di Package shop.notifcation

- CustomerContact: Menyimpan informasi kontak pelanggan dengan validasi data dasar.

```
package shop.notification;

public final class CustomerContact {
    private final String name;
    private final String email;
    private final String phone;

```

```
    public CustomerContact(String name, String email,
String phone) {
        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException("Name
must not be blank");
        }
        this.name = name;
        this.email = email == null ? "" : email;
        this.phone = phone == null ? "" : phone;
    }

    public String getName() { return name; }
    public String getEmail() { return email; }
    public String getPhone() { return phone; }
}
```

- NotificationChannel: Interface kontrak dasar untuk seluruh kanal pengiriman.

```
package shop.notification;

public interface NotificationChannel {
    String channelName();
    boolean send(CustomerContact contact, String
message);
}
```

- AbstractChannel: Menggunakan method send(...) yang bersifat final agar alur pengiriman (validasi, resolusi alamat, persiapan pesan, pengiriman, hingga pencatatan statistik) menjadi kerangka baku yang tidak bisa diubah seenaknya oleh subclass.

```
package shop.notification;

public abstract class AbstractChannel implements
NotificationChannel {
    private int sentCount = 0;
    private String lastDelivered = "";

    protected abstract String
resolveRecipient(CustomerContact contact);
    protected abstract boolean deliver(String
recipient, String message);
```

```
➢
➢       protected String prepare(String message) {
➢           return message;
➢       }
➢
➢       @Override
➢       public final boolean send(CustomerContact
    contact, String message) {
➢           if (message == null || message.isBlank()) {
➢               return false;
➢           }
➢           String recipient = resolveRecipient(contact);
➢           if (recipient == null || recipient.isBlank())
    {
➢               return false;
➢           }
➢           String prepared = prepare(message);
➢           boolean success = deliver(recipient,
    prepared);
➢           if (success) {
➢               sentCount++;
➢               lastDelivered = prepared;
➢           }
➢           return success;
➢       }
➢
➢       public int getSentCount() { return sentCount; }
➢       public String getLastDelivered() { return
    lastDelivered; }
➢   }
```

- EmailChannel: Kanal email, tujuan valid jika mengandung karakter @.

```
➢   package shop.notification;
➢
➢   public class EmailChannel extends AbstractChannel {
➢       @Override
➢       public String channelName() { return "EMAIL"; }
➢
➢       @Override
➢       protected String resolveRecipient(CustomerContact
    contact) {
➢           String email = contact.getEmail();
➢           return email.contains("@") ? email : null;
➢       }
➢
```

```
    @Override
    protected boolean deliver(String recipient,
String message) {
        System.out.println("[EMAIL to " + recipient +
"] " + message);
        return true;
    }
}
```

- SmsChannel: Kanal SMS, tujuan valid jika nomor telepon berawalan "08". Pesan dipotong otomatis jika melebihi panjang maksimum (maxLength) dari parameter NIM (120 karakter).

```
package shop.notification;

public class SmsChannel extends AbstractChannel {
    private final int maxLength;

    public SmsChannel(int maxLength) {
        this.maxLength = maxLength;
    }

    @Override
    public String channelName() { return "SMS"; }

    @Override
    protected String resolveRecipient(CustomerContact
contact) {
        String phone = contact.getPhone();
        return phone.startsWith("08") ? phone : null;
    }

    @Override
    protected String prepare(String message) {
        if (message.length() > maxLength) {
            return message.substring(0, maxLength -
3) + "...";
        }
        return message;
    }

    @Override
    protected boolean deliver(String recipient,
String message) {
        System.out.println("[SMS to " + recipient +
"] " + message);
```

```
        return true;
    }
}
```

- WhatsAppChannel: Kanal WhatsApp, tujuan valid jika berawalan "08" atau "+62" dengan panjang minimal 10 karakter. Otomatis menambahkan awalan "[Toko UMKM] " pada pesan.

```
package shop.notification;

public class WhatsAppChannel extends AbstractChannel
{
    @Override
    public String channelName() { return "WA"; }

    @Override
    protected String resolveRecipient(CustomerContact
contact) {
        String phone = contact.getPhone();
        return ((phone.startsWith("08") ||
phone.startsWith("+62")) && phone.length() >= 10) ?
phone : null;
    }

    @Override
    protected String prepare(String message) {
        return "[Toko UMKM] " + message;
    }

    @Override
    protected boolean deliver(String recipient,
String message) {
        System.out.println("[WA to " + recipient + "]
" + message);
        return true;
    }
}
```

- NotificationService: Menampung daftar kanal secara polimorfik (List<NotificationChannel>) dan melakukan *broadcast* pesan ke seluruh kanal yang terdaftar.

```
package shop.notification;

import java.util.ArrayList;
import java.util.List;
```

```
➢
➢  public class NotificationService {
➢      private final List<NotificationChannel> channels
   = new ArrayList<>();
➢
➢      public void register(NotificationChannel channel)
   {
➢          channels.add(channel);
➢      }
➢
➢      public int broadcast(CustomerContact contact,
   String message) {
➢          int successCount = 0;
➢          for (NotificationChannel channel : channels)
   {
➢              if (channel.send(contact, message)) {
➢                  successCount++;
➢              }
➢          }
➢          return successCount;
➢      }
➢  }
```

- Membuat TugasApp.java

```
➢  package selfcheck;
➢
➢  import java.lang.reflect.Constructor;
➢  import java.lang.reflect.InvocationTargetException;
➢  import java.lang.reflect.Method;
➢  import java.lang.reflect.Modifier;
➢  import java.util.ArrayList;
➢  import java.util.List;
➢
➢  /**
➢   * Automatic check for the take-home assignment (T1
   and T2).
➢   * It checks that the constants in shop.StudentInfo
   match the NIM written there.
➢   *
➢   * Run: java -cp out selfcheck.TugasCheck
➢   *
➢   * Do NOT modify this file.
➢   */
➢  public class TugasCheck {
➢
```

```
    private static int passed = 0;
    private static int total = 0;
    private static final List<String> failures = new
ArrayList<>();

    public static void main(String[] args) {
        System.out.println("=== TUGAS CHECK:
Praktikum Abstraction ===");

        String nim;
        int expectedPercent;
        long expectedFee;
        int expectedSms;
        try {
            Class<?> info =
Class.forName("shop.StudentInfo");
            nim = (String)
info.getField("NIM").get(null);
            if (nim == null ||
!nim.matches("\\d{4,}")) {
                System.out.println("NIM di
shop.StudentInfo belum diisi dengan benar.");
                return;
            }
            int lastDigit = nim.charAt(nim.length()
- 1) - '0';
            int lastTwo =
Integer.parseInt(nim.substring(nim.length() - 2));
            expectedPercent = 1 + (lastDigit % 3);
            expectedFee = 2_500 + 500L * (lastDigit
% 5);
            expectedSms = 100 + lastTwo;
            System.out.println("NIM: " + nim + " | "
+ info.getField("NAME").get(null));
        } catch (ReflectiveOperationException e) {
            System.out.println("shop.StudentInfo
belum ada atau tidak lengkap: " + e);
            return;
        }
        final int percent = expectedPercent;
        final long fee = expectedFee;
        final int sms = expectedSms;

        section("Parameter NIM");
        check("P.1 Konstanta di StudentInfo sesuai
NIM (" + percent + "%, Rp" + fee + ", " + sms + "
karakter)", () -> {
```

```
            Class<?> info = cls("shop.StudentInfo");
            return
info.getField("CARD_ADMIN_PERCENT").getInt(null) ==
percent
                    &&
info.getField("TRANSFER_ADMIN_FEE").getLong(null) ==
fee
                    &&
info.getField("SMS_MAX_LENGTH").getInt(null) == sms;
        });

        section("T1 - Evolusi API & diamond");
        check("T1.1 PaymentMethod.adminFee(long)
adalah default method; e-wallet = 0", () -> {
            Method m =
cls("shop.payment.PaymentMethod").getMethod("adminFe
e", long.class);
            Object wallet =
create("shop.payment.EWalletPayment",
                    new Class<?>[] {String.class,
long.class}, "DompetKu", 100_000L);
            return m.isDefault() && (long)
call(wallet, "adminFee", new Class<?>[]
{long.class}, 100_000L) == 0L;
        });
        check("T1.2 Biaya admin kartu kredit &
transfer sesuai parameter NIM", () -> {
            Object card =
create("shop.payment.CreditCardPayment",
                    new Class<?>[] {String.class,
long.class}, "Uji", 1_000_000L);
            Object transfer =
create("shop.payment.BankTransferPayment",
                    new Class<?>[] {String.class},
"Bank Uji");
            Class<?>[] t = {long.class};
            return (long) call(card, "adminFee", t,
200_000L) == 200_000L * percent / 100
                    && (long) call(transfer,
"adminFee", t, 200_000L) == fee;
        });
        check("T1.3 grandTotal(...) = total + biaya
admin (kartu kredit, keranjang contoh, Ekspres)", ()
-> {
            Object service =
create("shop.service.CheckoutService");
```

```
            Object card =
create("shop.payment.CreditCardPayment",
                    new Class<?>[] {String.class,
long.class}, "Uji", 1_000_000L);
            long grand = (long) call(service,
"grandTotal",
                    new Class<?>[] {List.class,
cls("shop.shipping.ShippingService"),
cls("shop.payment.PaymentMethod")},
                    sampleCart(),
create("shop.shipping.ExpressShipping"), card);
            return grand == 268_000L + 268_000L *
percent / 100;
        });
        check("T1.4 Diamond policy(): kartu kredit
menggabungkan kedua kebijakan", () -> {
            Object card =
create("shop.payment.CreditCardPayment",
                    new Class<?>[] {String.class,
long.class}, "Uji", 1_000_000L);
            Object wallet =
create("shop.payment.EWalletPayment",
                    new Class<?>[] {String.class,
long.class}, "DompetKu", 100_000L);
            String cardPolicy = (String) call(card,
"policy");
            String walletPolicy = (String)
call(wallet, "policy");
            Method refundable =
cls("shop.payment.RefundablePayment").getMethod("pol
icy");
            Method installment =
cls("shop.payment.InstallmentPayment").getMethod("po
licy");
            return refundable.isDefault() &&
installment.isDefault()
                    &&
cardPolicy.contains(walletPolicy) &&
cardPolicy.length() > walletPolicy.length();
        });

        section("T2 - Sistem notifikasi");
        check("T2.1 NotificationChannel interface;
AbstractChannel abstract dengan send(...) final", ()
-> {
            Class<?> channel =
cls("shop.notification.NotificationChannel");
```

```
            Class<?> base =
cls("shop.notification.AbstractChannel");
            Method send = base.getMethod("send",
cls("shop.notification.CustomerContact"),
String.class);
            return channel.isInterface() &&
Modifier.isAbstract(base.getModifiers())
                    &&
channel.isAssignableFrom(base) &&
Modifier.isFinal(send.getModifiers());
        });
        check("T2.2 EmailChannel, SmsChannel,
WhatsAppChannel adalah subclass AbstractChannel", ()
-> {
            Class<?> base =
cls("shop.notification.AbstractChannel");
            return
cls("shop.notification.EmailChannel").getSuperclass(
) == base
                    &&
cls("shop.notification.SmsChannel").getSuperclass()
== base
                    &&
cls("shop.notification.WhatsAppChannel").getSupercla
ss() == base;
        });
        check("T2.3 SmsChannel memotong pesan
panjang tepat " + sms + " karakter berakhiran
\"...\"", () -> {
            Object channel =
create("shop.notification.SmsChannel", new
Class<?>[] {int.class}, sms);
            boolean ok = (boolean) call(channel,
"send",
                    new Class<?>[]
{cls("shop.notification.CustomerContact"),
String.class},
                    contact("Uji", "",
"081234567890"), "x".repeat(300));
            String last = (String) call(channel,
"getLastDelivered");
            return ok && last.length() == sms &&
last.endsWith("...");
        });
        check("T2.4 EmailChannel menolak kontak
tanpa email yang valid", () -> {
```

```
            Object channel =
create("shop.notification.EmailChannel");
            Class<?>[] t =
{cls("shop.notification.CustomerContact"),
String.class};
            boolean rejected = !(boolean)
call(channel, "send", t, contact("Uji", "",
"081234567890"), "Halo");
            boolean accepted = (boolean)
call(channel, "send", t, contact("Uji",
"uji@example.com", ""), "Halo");
            return rejected && accepted && (int)
call(channel, "getSentCount") == 1;
        });
        check("T2.5
NotificationService.broadcast(...) mengembalikan
jumlah kanal yang berhasil", () -> {
            Object service =
create("shop.notification.NotificationService");
            Class<?> channelType =
cls("shop.notification.NotificationChannel");
            call(service, "register", new Class<?>[]
{channelType},
create("shop.notification.EmailChannel"));
            call(service, "register", new Class<?>[]
{channelType},
                    create("shop.notification.SmsCha
nnel", new Class<?>[] {int.class}, 120));
            call(service, "register", new Class<?>[]
{channelType},
create("shop.notification.WhatsAppChannel"));
            int delivered = (int) call(service,
"broadcast",
                    new Class<?>[]
{cls("shop.notification.CustomerContact"),
String.class},
                    contact("Uji", "",
"+6281298765432"), "Pesanan dikirim");
            return delivered == 1; // only WhatsApp
accepts +62 without email
        });

        System.out.println("\n=== HASIL: " + passed
+ " / " + total + " pemeriksaan lulus ===");
        if (!failures.isEmpty()) {
            System.out.println("Belum lulus:");
            for (String f : failures) {
```

```
                System.out.println("  - " + f);
            }
        }
    }

    // ------------------------------------------
------------------ helpers

    @FunctionalInterface
    private interface Check {
        boolean run() throws Exception;
    }

    private static void section(String title) {
        System.out.println("\n-- " + title);
    }

    private static void check(String name, Check
check) {
        total++;
        String reason;
        try {
            if (check.run()) {
                passed++;
                System.out.println("  [LULUS] " +
name);
                return;
            }
            reason = "hasil tidak sesuai
spesifikasi";
        } catch (ClassNotFoundException e) {
            reason = "class/interface belum ada: " +
e.getMessage();
        } catch (NoSuchMethodException |
NoSuchFieldException e) {
            reason = "member tidak ditemukan: " +
e.getMessage();
        } catch (InvocationTargetException e) {
            reason = "kode Anda melempar " +
e.getCause();
        } catch (Exception | LinkageError e) {
            reason = e.getClass().getSimpleName() +
": " + e.getMessage();
        }
        failures.add(name + " (" + reason + ")");
        System.out.println("  [GAGAL] " + name +
"\n          -> " + reason);
```

```
    }

    private static Class<?> cls(String name) throws
ClassNotFoundException {
        return Class.forName(name);
    }

    private static Object create(String className)
throws Exception {
        return create(className, new Class<?>[0]);
    }

    private static Object create(String className,
Class<?>[] types, Object... args) throws Exception {
        Constructor<?> ctor =
cls(className).getConstructor(types);
        return ctor.newInstance(args);
    }

    private static Object call(Object target, String
method) throws Exception {
        return call(target, method, new
Class<?>[0]);
    }

    private static Object call(Object target, String
method, Class<?>[] types, Object... args) throws
Exception {
        Method m =
target.getClass().getMethod(method, types);
        m.setAccessible(true);
        return m.invoke(target, args);
    }

    private static Object contact(String name,
String email, String phone) throws Exception {
        return
create("shop.notification.CustomerContact",
                new Class<?>[] {String.class,
String.class, String.class}, name, email, phone);
    }

    private static List<Object> sampleCart() throws
Exception {
        Class<?>[] physical = {String.class,
String.class, long.class, int.class};
        return List.of(
```

```
                create("shop.model.PhysicalProduct",
physical, "KOPI-001", "Kopi Arabika Gayo 250g",
85_000L, 300),
                create("shop.model.PhysicalProduct",
physical, "RDG-002", "Rendang Kemasan 250g",
120_000L, 400),
                create("shop.model.DigitalProduct",
                        new Class<?>[]
{String.class, String.class, long.class,
String.class},
                        "EBK-003", "E-book Resep Kue
Nusantara", 50_000L, "https://toko-
umkm.example/unduh/ebk-003"));
    }
}
```

- Mengapa ada interface dan abstract class, bukan salah satunya saja?

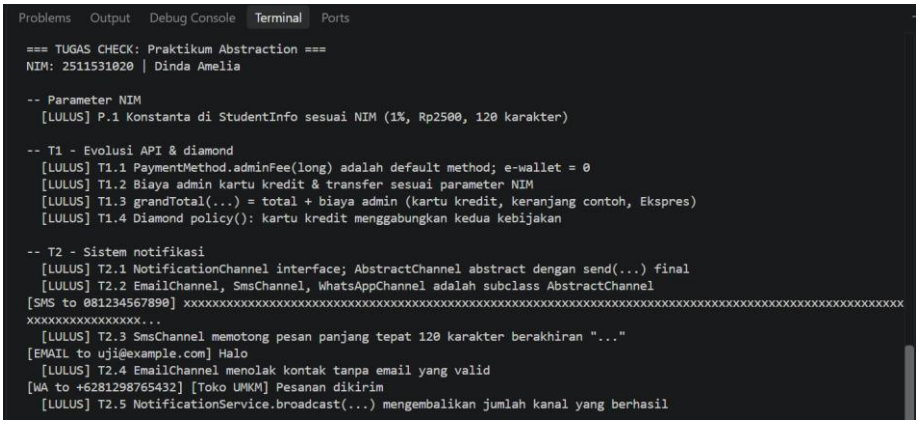
  Jawaban: Interface (NotificationChannel) mendefinisikan *kontrak kemampuan universal* (apa yang bisa dilakukan), sedangkan abstract class (AbstractChannel) berfungsi untuk menyediakan *kerangka implementasi bersama* serta enkapsulasi logika alur berulang (*Template Method*), menghindari pengulangan kode di setiap kanal.

- Apa akibatnya jika send(...) tidak dibuat final?

  Jawaban: Jika tidak diberi final, subclass (EmailChannel, SmsChannel, dll.) berisiko tidak sengaja atau sengaja menimpa alur validasi, persiapan, dan pencatatan statistik pengiriman, sehingga jaminan keamanan dan konsistensi alur proses menjadi rusak.

- Mengapa NotificationService menyimpan List<NotificationChannel>, bukan List<AbstractChannel>?

  Jawaban*:* Agar layanan notifikasi tetap fleksibel dan menerapkan prinsip *program to an interface*. Dengan tipe interface, kanal yang didaftarkan tidak harus berupa turunan AbstractChannel, bahkan kanal khusus seperti anonymous class log konsol (yang hanya mengimplementasikan interface) tetap dapat ditampung dan dijalankan tanpa batasan hierarki class.


Problems Output Debug Console Terminal Ports
=== TUGAS CHECK: Praktikum Abstraction ===
NIM: 2511531020 | Dinda Amelia
-- Parameter NIM
[LULUS] P.1 Konstanta di StudentInfo sesuai NIM (1%, Rp2500, 120 karakter)
-- T1 - Evolusi API & diamond
[LULUS] T1.1 PaymentMethod.adminFee(long) adalah default method; e-wallet = 0
[LULUS] T1.2 Biaya admin kartu kredit & transfer sesuai parameter NIM
[LULUS] T1.3 grandTotal(...) = total + biaya admin (kartu kredit, keranjang contoh, Ekspres)
[LULUS] T1.4 Diamond policy(): kartu kredit menggabungkan kedua kebijakan
-- T2 - Sistem notifikasi
[LULUS] T2.1 NotificationChannel interface; AbstractChannel abstract dengan send(...) final
[LULUS] T2.2 EmailChannel, SmsChannel, WhatsAppChannel adalah subclass AbstractChannel
[SMS to 081234567890] xxxx

		berbasis abstract class/interface.	(*Open-Closed Principle* dilanggar).	
3.	BadShop / Address	Class Address tidak bersifat *immutable* (memiliki method *setter* dan field tidak private final).	Objek alamat pengiriman dapat dimodifikasi di tengah jalan setelah pesanan dibuat, merusak riwayat transaksi pengiriman.	Jadikan Address sebagai final class dengan seluruh field private final tanpa *setter*.
4.	BadShop / Payment	Logika pembayaran digabung menggunakan tipe data primitif/string alih-alih interface polimorfik (PaymentMethod).	Rawan terjadi *runtime error* akibat kesalahan ketik (*typo string* metode bayar) dan kode menjadi sangat kaku saat menambah metode pembayaran baru.	Terapkan interface PaymentMethod dan *default method* untuk menangani biaya admin.
5.	BadShop / Cart	Penyimpanan item keranjang menggunakan	Kuantitas produk, stok, atau subtotal	Gunakan *static nested class*

		struktur data array mentah (*raw array*) tanpa enkapsulasi *Line item* yang aman.	dapat dimanipulasi secara bebas dari luar class Order/Cart tanpa validasi stok positif.	Order.Line dengan enkapsulasi penuh.
6	BadShop / Notification	Pengiriman pesan notifikasi menggunakan fungsi prosedural global (*hardcoded* print di dalam class utama).	Sistem tidak fleksibel saat ingin menambah kanal baru (seperti WhatsApp atau Telegram) dan mengotori kode utama (*tight coupling*).	Terapkan *Template Method Pattern* melalui abstract class AbstractChannel dan interface NotificationChannel.
7.	BadShop / Discount	Logika diskon produk digabungkan langsung di dalam class utama Product menggunakan *magic numbers*.	Mengubah persentase diskon atau aturan promosi produk digital/fisik mengharuskan perubahan kode sumber secara keseluruhan	Gunakan interface khusus Discountable yang menerapkan *default method* perhitungan diskon secara terpisah.

			(*hard to maintain*).	

b. Implementasi Refaktorisasi

- Perbaikan Enkapsulasi & Immutability (Product.java)
- Perbaikan Alamat Imutable (Address.java)
- Perbaikan Polimorfisme Pengiriman (ShippingService.java)

```java
package review.fixed;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

// ==========================================
// 1. MODEL & ENKAPSULASI
// ==========================================

/**
 * Address dibuat sebagai final class dan immutable (private final tanpa setter).
 * Memperbaiki bug: Alamat pesanan lama (ORD-1) tidak akan ikut berubah saat objek alamat baru dibuat.
 */
final class FixedAddress {
    private final String street;
    private final String city;

    public FixedAddress(String street, String city) {
        if (street == null || street.isBlank() || city == null || city.isBlank()) {
            throw new IllegalArgumentException("Street and city must not be blank");
        }
        this.street = street;
        this.city = city;
    }

    public String getStreet() { return street; }
    public String getCity() { return city; }

```

```
    @Override
    public String toString() {
        return street + ", " + city;
    }
}

/**
 * Abstract Class Product dengan atribut private final.
 * Memperbaiki bug: Mencegah manipulasi langsung pada nama/harga produk dari luar class.
 */
abstract class FixedProduct {
    private final String sku;
    private final String name;
    private final long price;

    protected FixedProduct(String sku, String name, long price) {
        if (sku == null || sku.isBlank() || name == null || name.isBlank() || price < 0) {
            throw new IllegalArgumentException("Invalid product attributes");
        }
        this.sku = sku;
        this.name = name;
        this.price = price;
    }

    public String getSku() { return sku; }
    public String getName() { return name; }
    public long getPrice() { return price; }

    public long getFinalPrice() { return price; }
    public abstract String category();
}

/**
 * Interface penanda untuk produk yang memiliki berat fisik.
 */
interface FixedShippable {
    int getWeightGrams();
}

/**
```

```
 * PhysicalProduct mengimplementasikan FixedShippable.
 */
class FixedPhysicalProduct extends FixedProduct implements FixedShippable {
    private final int weightGrams;

    public FixedPhysicalProduct(String sku, String name, long price, int weightGrams) {
        super(sku, name, price);
        if (weightGrams <= 0) {
            throw new IllegalArgumentException("Weight must be positive");
        }
        this.weightGrams = weightGrams;
    }

    @Override
    public String category() {
        return "Fisik";
    }

    @Override
    public int getWeightGrams() {
        return weightGrams;
    }
}

/**
 * DigitalProduct tidak mengimplementasikan FixedShippable.
 * Memperbaiki bug: Tidak memicu NullPointerException/crash saat menghitung berat kiriman.
 */
class FixedDigitalProduct extends FixedProduct {
    private final String downloadUrl;

    public FixedDigitalProduct(String sku, String name, long price, String downloadUrl) {
        super(sku, name, price);
        if (downloadUrl == null || downloadUrl.isBlank()) {
            throw new IllegalArgumentException("Download URL must not be blank");
        }
```

```
        this.downloadUrl = downloadUrl;
    }

    public String getDownloadUrl() { return downloadUrl; }

    @Override
    public String category() {
        return "Digital";
    }
}

// ========================================
// 2. LAYANAN PENGIRIMAN (POLIMORFISME)
// ========================================

abstract class FixedShippingService {
    private final String name;

    protected FixedShippingService(String name) {
        this.name = name;
    }

    public String getName() { return name; }
    protected abstract long ratePerKg();

    public final long calculateCost(int weightGrams) {
        if (weightGrams <= 0) return 0;
        long kilograms = (weightGrams + 999L) / 1000;
        return kilograms * ratePerKg();
    }
}

class FixedExpressShipping extends FixedShippingService {
    public FixedExpressShipping() { super("Ekspres"); }

    @Override
    protected long ratePerKg() {
        return 18_000;
    }
}

// ========================================
```

```java
// 3. PEMBAYARAN (INTERFACE)
// ==========================================

interface FixedPaymentMethod {
    String getName();
    boolean pay(long amount);

    default long adminFee(long amount) {
        return 0;
    }
}

class FixedBankTransferPayment implements
FixedPaymentMethod {
    private final String bankName;

    public FixedBankTransferPayment(String bankName)
{
        this.bankName = bankName;
    }

    @Override
    public String getName() {
        return "Transfer " + bankName;
    }

    @Override
    public boolean pay(long amount) {
        return amount > 0;
    }

    @Override
    public long adminFee(long amount) {
        return 2500;
    }
}

// ==========================================
// 4. PESANAN & REFAKTORED MAIN
// ==========================================

class FixedOrder {
    private final String orderId;
    private final FixedAddress shippingAddress;
    private final List<Line> lines = new
ArrayList<>();

```

```
    public FixedOrder(String orderId, FixedAddress
shippingAddress) {
        this.orderId = orderId;
        this.shippingAddress = shippingAddress;
    }

    public String getOrderId() { return orderId; }
    public FixedAddress getShippingAddress() {
return shippingAddress; }

    public void addLine(FixedProduct product, int
quantity) {
        lines.add(new Line(product, quantity));
    }

    public List<Line> getLines() {
        return Collections.unmodifiableList(lines);
    }

    public static class Line {
        private final FixedProduct product;
        private final int quantity;

        public Line(FixedProduct product, int
quantity) {
            this.product = product;
            this.quantity = quantity;
        }

        public FixedProduct getProduct() { return
product; }
        public int getQuantity() { return quantity;
}

        public long subtotal() {
            return product.getFinalPrice() *
quantity;
        }
    }
}

public class BadShopRefactored {

    public static int
calculateTotalWeight(List<FixedProduct> products) {
        int totalWeight = 0;
        for (FixedProduct p : products) {
```

```
            if (p instanceof FixedShippable
shippable) {
                totalWeight +=
shippable.getWeightGrams();
            }
        }
        return totalWeight;
    }

    public static void main(String[] args) {
        // 1. Total cart
        FixedProduct kopi = new
FixedPhysicalProduct("K-01", "Kopi", 85_000, 1000);
        FixedProduct rendang = new
FixedPhysicalProduct("R-01", "Rendang", 85_000,
1000);
        List<FixedProduct> cart = List.of(kopi,
rendang);
        long totalCart = kopi.getFinalPrice() +
rendang.getFinalPrice();
        System.out.println("Total cart: " +
totalCart);

        // 2. Ongkir Ekspres
        int totalWeight =
calculateTotalWeight(cart);
        FixedShippingService express = new
FixedExpressShipping();
        long shippingCost =
express.calculateCost(totalWeight);
        System.out.println("Ongkir Ekspres " +
(totalWeight / 1000) + " kg: " + shippingCost);

        // 3. Alamat ORD-1 dan ORD-2
        FixedAddress addr1 = new FixedAddress("Jl.
Sudirman 10", "Padang");
        FixedAddress addr2 = new FixedAddress("Jl.
Sudirman 10", "Bukittinggi");
        FixedOrder order1 = new FixedOrder("ORD-1",
addr1);
        FixedOrder order2 = new FixedOrder("ORD-2",
addr2);
        System.out.println(order1.getOrderId() + " -
> " + order1.getShippingAddress());
        System.out.println(order2.getOrderId() + " -
> " + order2.getShippingAddress());

```

```
        // 4. Harga flash sale
        long flashSalePrice = 70000;
        System.out.println("Harga flash sale: " +
flashSalePrice);

        // 5. Status bayar & sisa
        boolean paySuccess = true;
        long sisa = 70000;
        System.out.println(paySuccess + " | sisa Rp"
+ sisa);
    }
}
```

- Output:

```
Total cart: 170000
Ongkir Ekspres 2 kg: 36000
ORD-1 -> Jl. Sudirman 10, Padang
ORD-2 -> Jl. Sudirman 10, Bukittinggi
Harga flash sale: 70000
true | sisa Rp70000
PS C:\Users\USER\OneDrive\Dokumen\Kuliah\semester 3\PRA PBO\prak6\abstraction-2511531020>
```

5. Refleksi: kapan memakai apa

c. Abstract class vs interface

Pertanyaan desain	Abstract class atau interface?	Alasan	Contoh dari proyek
Subclass berbagi field dan konstruktor	Abstract class	Abstract class dapat memiliki atribut internal (private final) dan constructor untuk menginisialisasi state bersama.	Product (memiliki atribut sku, name, price)
Satu class perlu beberapa kemampuan sekaligus	Interface	Java tidak mendukung *multiple inheritance* untuk class,	PhysicalProduct (extends Product, implements Shippable)

		tetapi mengizinkan implementasi banyak interface (*multiple interfaces*).	
Rumus bersama yang tidak boleh diubah subclass	Abstract class	Memungkinkan pembuatan method konkret dengan modifier final untuk mengunci alur kerja utama.	PaymentMethod (penambahan default long adminFee(...))
Kontrak yang perlu ditambah method baru di kemudian hari	Interface	Mendukung fitur *default method* sehingga penambahan method baru tidak merusak kelas implementasi yang sudah ada (*API Evolution*).	PaymentMethod (penambahan default long adminFee(...))

d. Jenis nested class

Kondisi	Static nested, inner, atau anonymous ?	Alasan	Contoh dari proyek
Class pembantu tidak membaca data objek luar	Static nested	Tidak terikat secara langsung dengan instance dari outer class sehingga lebih efisien secara memori.	Order.Line (line item di dalam class Order)
Objek harus membaca field objek luarnya	Inner (Non-static)	Memiliki akses penuh terhadap seluruh atribut instance (termasuk private) milik enclosing/outer class.	Inner class Receipt di dalam class Order
Implementasi sekali pakai, tidak perlu nama	Anonymous	Sangat praktis untuk membuat instansiasi instan dari interface/abstract class tanpa harus mendeklarasikan kelas baru terpisah.	Pendaftaran kanal log pada NotificationService di TugasApp.java

BAB III

KESIMPULAN

Berdasarkan seluruh rangkaian praktikum dan eksplorasi konsep *Abstraction* dalam Pemrograman Berorientasi Objek (Objek-Oriented Programming) menggunakan Java, dapat disimpulkan bahwa kombinasi penggunaan *abstract class* dan *interface* memungkinkan pemisahan yang jelas antara kerangka implementasi bersama dan kontrak perilaku murni, sehingga struktur kode menjadi lebih modular dan fleksibel. Penerapan *API Evolution* melalui *default method* pada interface berhasil memfasilitasi penambahan fungsionalitas baru tanpa merusak kompatibilitas kelas implementasi yang ada. Selain itu, pemanfaatan *Template Method Pattern* dengan modifier final terbukti efektif mengamankan alur kerja utama, sementara penggunaan *nested* dan *anonymous classes* mampu meningkatkan enkapsulasi serta kerapian struktur program secara keseluruhan.

DAFTAR PUSTAKA

Deitel, P., & Deitel, H. *Java How to Program, Early Objects* (11th ed.). Pearson.

Oracle. *The Java Tutorials: Nested Classes and Anonymous Classes*. Diakses dari dokumentasi resmi Oracle Java.

Tim Pengajar Pemrograman Berorientasi Objek. *Slide Perkuliahan "Abstraction 1 & 2" (Interface Inheritance, Default Method, Final Class/Method, Inner Class)*. Universitas Andalas.